

Objective

LLMs are great at general knowledge, but your most critical answers live in your data.

The objective is to design and deploy AI that securely taps your sensitive, domain-specific datasets so decisions are accurate, auditable, and ready for production.

Solution

Databricks now offers **end-to-end AI and agentic solutions** — from **data ingestion & governance** to **LLM serving, RAG pipelines, multi-agent orchestration, and compliance tooling**.

Why Databricks is Suitable for Agents

- Databricks has a set of capabilities that make it a good foundation for building both ML-driven and LLM-powered “agents.”
- Databricks is suitable for agent development because it combines **data, ML, orchestration, and deployment** in one platform.
- The need to design the agent logic — Databricks doesn’t “do it for you” — but it removes much of the plumbing work. Here’s a **consolidated table** capturing all the **core capabilities of Databricks for building and running agents**, integrating your latest points with the previous architecture/lifecycle concepts:

Capability Area	Key Features & Best Practices
Unified Data & Compute Platform	- Delta Lake with ACID guarantees for reliable structured & semi-structured data. - Single environment for ETL, feature engineering, and analytics — no need to stitch multiple tools. - Auto Loader + streaming for near real-time event ingestion.
Built-in ML & Model Ops	- MLflow for tracking, versioning, and serving models. - Feature Store for consistent feature reuse across models and agents. - Databricks Model Serving for deploying API endpoints agents can call.
Language & Framework Flexibility	- Develop agents in Python, Scala, R etc. - Integrate with LangChain / LlamaIndex for LLM orchestration. - Support for Hugging Face / Mosaic AI for LLM fine-tuning & inference. - Native REST API integrations for external calls.
Orchestration & Workflows	- Databricks Workflows for orchestrating notebooks, jobs, pipelines. - Integration with external orchestrators and event triggers (Kafka, Kinesis). - Supports multi-agent orchestration (LangChain multi-agent manager or custom).

Scalability & Cost Control	- Cloud-native & cluster-based: scale compute up/down on demand. - Separate dev/prod environments with role-based access. - Support for batch, interactive, and real-time workloads.
Unstructured Data & Vector Search	- Store & process unstructured data (PDFs, notes, policies). - Build RAG pipelines leveraging Delta + embeddings. - Integrate with Milvus, Pinecone, Weaviate or Mosaic AI Vector Search for vector search.
Enterprise Security & Integration	- Role-based access control (RBAC), audit logs, VPC networking. - Easy integration with enterprise systems for compliance & governance.

Databricks + LangGraph

Abstract

To make AI agents truly useful inside an enterprise, the **data foundation (Databricks)** and **reasoning/orchestration (LangGraph)** must work hand in hand.

- **Databricks:** Unified data & ML foundation (features, models, embeddings, pipelines, governance).
- **LangGraph:** Multi-agent orchestration (LLM + SLM + tools).
- **Continuous Learning Loop:** Feedback → Prompt tuning → RAG refresh → Fine-tuning → Safe redeployment.
- **Outcome:** Agents that evolve, remain accurate, cost-efficient, and aligned with business goals.

Data Foundation – Lakehouse for All Data Needs

Component	Purpose / Role	Key Features & Best Practices
Delta Lake	Central store for all enterprise data	- Stores raw + curated structured, semi-structured, unstructured data - ACID transactions, time-travel, scalable storage - Single source of truth for agents, ML, and analytics
Unity Catalog	Enterprise-grade governance	- Fine-grained access control across all data and ML assets - Lineage & audit logging for compliance - Unified governance across ETL, ML, BI workloads

ETL / ELT Pipelines	Data ingestion & transformation	- Delta Live Tables for declarative, streaming + batch pipelines - Apache Spark for large-scale transformations - Continuous or scheduled ingestion from internal/external sources
Feature Store	Curated ML features	- Central repository of real-time and batch features for ML models and agents - Ensures consistency and reusability of features across teams and use cases
Vector Search	Semantic retrieval for LLM-based agents	- Native vector database for storing embeddings - Enables RAG (Retrieval-Augmented Generation) with context retrieval - Works with unstructured text (policies, call notes, PDFs)
BI & Analytics	Human decision support & reporting	- SQL Warehouses for analytics queries - Dashboards and visualization tools on curated data - Can be integrated with agent insights for end-user visibility

Agentic / AI Solutions

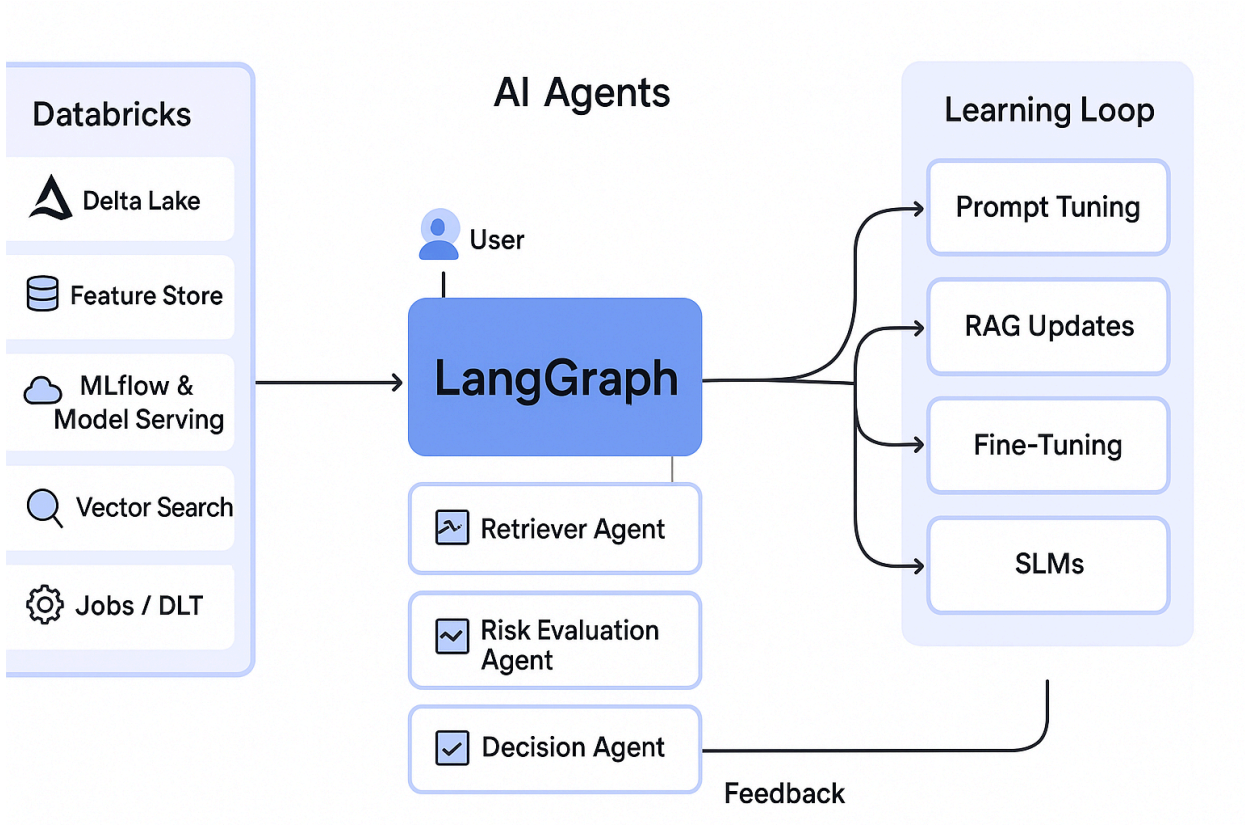
Capability	Purpose / Role	Key Features & Best Practices
Model Serving	Host and expose ML/LLM models via APIs	- Low-latency serving of foundation LLMs (e.g., DBRX) , fine-tuned SLMs , and task-specific models .- Databricks Model Serving or MLflow for deployment.- Supports versioning, scaling, inference logging.
RAG-native	Ground LLM outputs with enterprise data	- Combine private enterprise data with LLMs using Vector Search .- Enables Retrieval-Augmented Generation (RAG) for factual, context-aware responses.
LangChain / LangGraph Integrations	Multi-step, tool-using agent orchestration	- Build deterministic multi-step agents .- Agents can call SQL, APIs, ML models , and other agents .- Provides control flow and memory handling.
MCP (Model Context Protocol)	Secure and controlled agent-tool interaction	- Standardized protocol for secure access to external tools.- Examples: SQL queries, approval systems, CRM APIs .- Helps maintain governance and safety boundaries.
SLMs Orchestration +	Lightweight control and validation agents	- Smaller, domain-tuned models for routing, validation, and safety checks.- Optimizes cost and improves reliability.- Combines rule-based logic with ML reasoning.

Workflow Automation	Orchestrate complex agent flows	- Databricks Jobs & Workflows for scheduling and orchestration.- Integrates with Kafka / API Gateway for event-driven real-time triggers.- Useful for multi-agent coordination and end-to-end pipelines.
----------------------------	---------------------------------	--

Architecture

Flow:

1. User interacts with an agent → LangGraph routes query to appropriate tools.
2. Agents call Databricks APIs (feature queries, RAG retrieval, ML predictions).
3. Decision/reply returned with reasoning & citations.



Learning Loop (Continuous Improvement)

For agents to improve over time, add a **feedback + retraining loop**:

Stage	Purpose / Role	Key Activities & Best Practices
Capture	Log agent activity for observability	- Record all agent inputs/outputs , tool usage, and outcomes into Delta tables .- Enables traceability and post-hoc analysis.
Feedback	Gather explicit and implicit signals	- Explicit : thumbs up/down, SME reviews.- Implicit : business KPIs (e.g., approval rates), fraud flags, error logs.- Store in Delta for unified analysis.
Aggregate & Analyze	Identify failure patterns and gaps	- Use Databricks pipelines to detect where agents fail.- Analyze knowledge gaps, prompt errors, safety violations .- Build dashboards for ongoing monitoring.
Update Models/Logic	Continuous agent improvement via 3 levers	Prompt Tuning :- Adjust prompts/templates in LangGraph for fast fixes without retraining.- Example: better citations in underwriter agents. RAG Updates :- Re-index and refresh embeddings in Databricks Vector Search .- Keeps knowledge current (e.g., policy changes, FAQs). Fine-Tuning :- Train adapters or full model fine-tunes using MLflow pipelines .- Use LoRA/distillation for domain-specific improvements (fraud terms, policies).
Evaluate & Deploy	Safely roll out improvements	- Validate with test sets and shadow/canary deployments .- Promote updated versions in Databricks Model Serving .- Monitor performance post-deployment.
Close the Loop	Continuous governance & lineage	- Maintain full lineage in Unity Catalog .- Monitor models and agents over time for compliance and drift.- Ensure updates are auditable and governed.

Adding SLMs (Small Language Models)

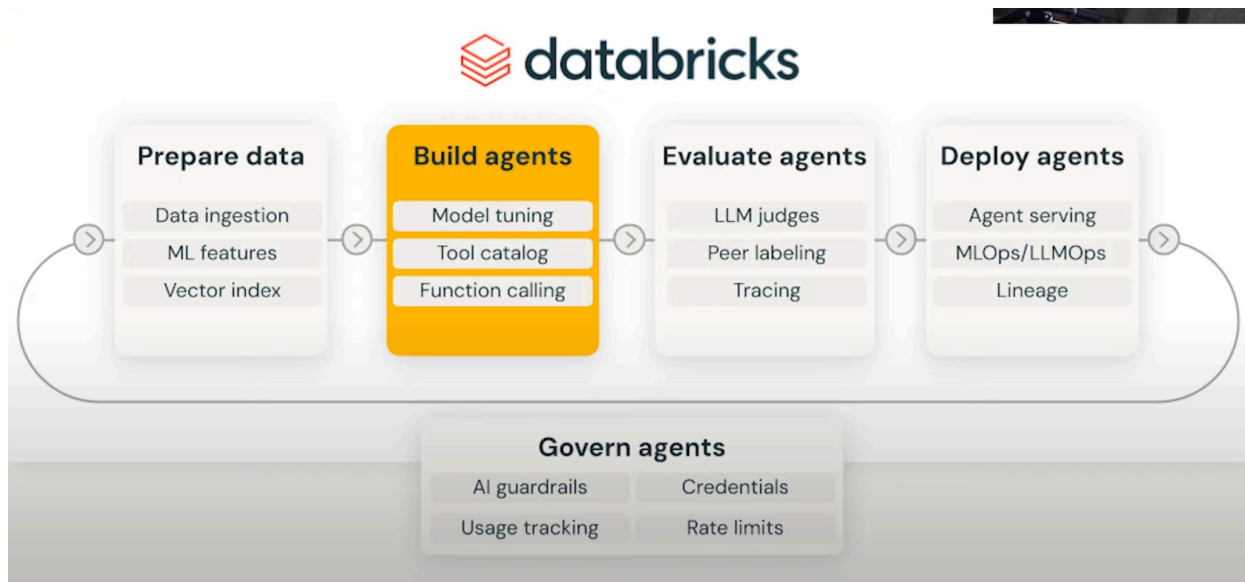
SLMs play a big role alongside LLMs:

- **Why**: Lower cost, faster inference, easier to host on-prem.
- **Where**: Routine tasks (classification, simple decisions, summarization).
- **How**:
 - LangGraph routes simple queries to SLMs, complex queries to LLMs.
 - Fine-tune SLMs on enterprise data for narrow tasks.
 - Even SLMs can use RAG for fresh knowledge.

- **Feedback Loop:**
Track performance of both SLMs and LLMs, optimize routing, retrain SLMs to handle more load cost-effectively.

Agent Lifecycle on Databricks

Agent Life cycle in databricks

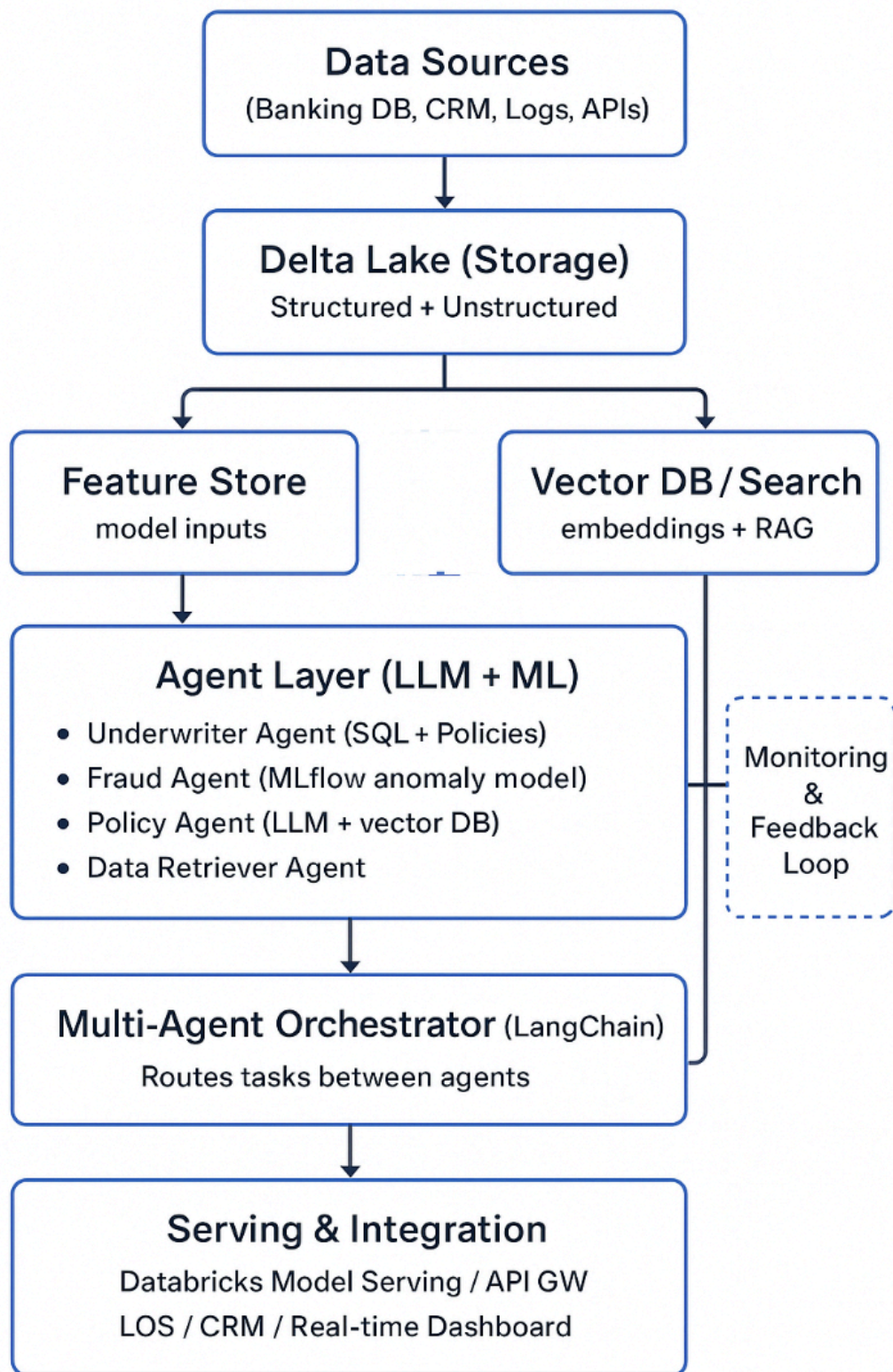


Stage	Goal	Key Activities / Features
Prepare Data	Ingest, centralize, and secure enterprise data	- Ingest structured & unstructured data into Delta Lake - Apply security & access control via Unity Catalog - Enable analytics-ready and AI-ready datasets
Build Agents	Develop domain-specific intelligent agents	- Start with LLMs/SLMs- Enhance with proprietary data (RAG, fine-tuning, prompt tuning) - Orchestrate workflows with LangChain / LangGraph / Mosaic AI
Evaluate Agents	Validate quality, accuracy, and safety	- Test against ground-truth Q&A datasets - Continuously expand evaluation datasets - Assess accuracy, latency, compliance- Include HITL (Human-In-The-Loop) reviews

Deploy Agents	Serve and manage agents at scale	- Register & version in MLflow - Governance & lineage via Unity Catalog - Deploy using Mosaic AI Model Serving Endpoints with: • API endpoints • Traffic splitting • Inference logging & metrics - AI Gateway for rate limiting, safety & content filtering- SME review apps to capture feedback
Govern & Improve	Ensure trust, compliance, and continuous improvement	- Unified governance for all use cases - Monitor metrics & feedback loops - Maintain audit trails & regulatory compliance - Continuous iteration on data, prompts, and evaluations

Example: Underwriter Assistant Agent

Component	Purpose / Best Practices / Examples
Data Layer (Delta Lake)	- Store structured & unstructured data (customer profiles, transactions, call center notes). - Use Delta tables for versioned, queryable data with ACID compliance. - Ingest using Auto Loader / Delta Live Tables for scalable ETL.
Feature & Knowledge Layer	- Feature Store: Share ML features (risk scores, behavioral features) across agents. - Vector Store: For LLM retrieval (policies, knowledge base). Options: Databricks + Milvus/Weaviate/Pinecone or Mosaic AI Vector Search if available.
Agents (LLMs + Tools)	- Build using LangChain / LlamaIndex inside Databricks notebooks. - Types: Data Retriever, Policy Reasoning, Fraud Detection, Decision Recommender. - Agents can call: SQL queries, APIs (banking/risk APIs), Custom ML models (via MLflow). - Combine rule-based + LLM reasoning; extend to multi-agent workflows.
Orchestration & Communication	- Use LangChain Multi-Agent Manager or custom orchestrator. - Specialized agents (e.g., Underwriting Agent, Fraud Agent, Policy Agent). - Databricks Workflows for batch/scheduled orchestration. - For real-time: use API Gateway or Kafka triggers for event-driven workflows.
Serving Layer	- Deploy via Databricks Model Serving (serverless endpoints). - Alternative: MLflow Models + API Gateway for external system integration. - Built-in features: traffic splitting, inference logging, rate limiting, safety filters.



Component	Details / Examples
Data Sources	Examples: Core Banking DB, CRM, Transaction Logs, External APIs Purpose: Continuous ingestion of structured (tables), semi-structured (JSON/CSV), unstructured (PDFs, notes) Best Practices: Use Databricks Auto Loader or Delta Live Tables for scalable ETL
Agent Types	Examples: - Data Retriever Agent: Fetches info from Delta tables - Policy Reasoning Agent: Uses LLM to interpret policies - Fraud Detection Agent: Calls ML anomaly detection models - Decision Recommender Agent: Synthesizes responses for the underwriter Flow: LLM queries Delta tables, combines rule-based & LLM reasoning; extendable to multiple agents
Multi-Agent Orchestration	Examples: - Fraud Agent: Calls ML model via MLflow - Policy Agent: Uses LLM & vector DB for policies - Aggregator Agent: Summarizes and recommends Pattern: Agents can call each other for complex workflows
Real-Time Deployment	- Deploy agents as Databricks Model Serving endpoints - Connect to LOS/CRM via APIs or Kafka for event-driven triggers
Scaling & Continuous Improvement	- Databricks Workflows for periodic retraining - Feature Store for shared features across agents - Vector Search + RAG to enrich LLM context

Demonstration

Here's a clean **flow description** for the underwriter query scenario:

1. Underwriter Query Initiation
 - Input: *"Can I approve customer 12345? What are the risks?"*
2. Data Retrieval
 - **Data Agent** fetches customer profile and repayment history from **Delta tables**.
3. Fraud Assessment
 - **Fraud Agent** calls an ML anomaly detection model hosted via **MLflow** to compute fraud risk.
4. Policy Evaluation

- **Policy Agent** performs **RAG** against a **vector database** of policy documents.
- Uses an **LLM** to interpret relevant policy conditions.
- 5. Orchestration
 - **LangChain (or LangGraph)** orchestrates the agents, each acting as a **tool/sub-agent**. Combines results from Data, Fraud, and Policy Agents.
- 6. Decision Synthesis
 - **Decision Agent** consolidates all inputs into an actionable answer.
- 7. Output to Underwriter
 - Example: *"Customer 12345: Low fraud risk, meets income and credit policy. Approve."*

Real-Time Deployment Outline

1. Data Streaming / Event Trigger:

- Use **Kafka / AWS Kinesis / Azure Event Hub** to push loan applications to Databricks.
- Each event triggers a Databricks Model Serving endpoint.

2. Endpoint Setup:

- Package agents as a REST API with LangChain inside Databricks Model Serving.
- Each call processes customer ID, fetches data, runs agents, and returns a recommendation.

3. Low Latency Storage:

- Use **Delta Live Tables** with Auto-Loader for ingestion.
- Keep frequently used embeddings in a low-latency vector DB.

4. Agent Orchestration:

- Use **LangChain Agents** inside a Databricks notebook or Databricks Workflows (for scheduled runs).

- For high throughput, consider deploying the orchestration layer as a **serverless API** (e.g., Databricks + API Gateway).

5. Integration with LOS / CRM:

- Call the API from your LOS or CRM.
- Push decision and rationale back to UI.